

PV Charge Control – Projekt Blueprint

Ziel

Ein einfacher, robuster Python-Service auf einem Raspberry Pi zur PV-Überschussregelung.

Komponenten:

- SolaX X3 Wechselrichter via Modbus TCP
- Eassee Home Wallbox
- optional MQTT
- systemd-Service
- kein Webserver
- single-threaded
- synchrones Polling

Vollständige Projektstruktur

```
pv-charge-control/  
├── main.py  
├── config.yaml  
├── requirements.txt  
├── README.md  
├── .env  
├──  
├── pvcharge/  
│   ├── __init__.py  
│   ├── config.py  
│   ├── controller.py  
│   ├── mqtt.py  
│   ├── models.py  
│   ├── util.py  
│   ├── averaging.py  
│   └── logging_setup.py  
├──  
├── inverter/  
│   ├── __init__.py  
│   ├── base.py  
│   └── solax_modbus.py  
├──  
└── wallbox/  
    ├── __init__.py  
    ├── base.py  
    ├── eassee_local.py  
    └── eassee_cloud.py
```

```
|
|
|— scripts/
|   |— install.sh
|   |— create_venv.sh
|
|— systemd/
|   |— pv-charge-control.service
|
|— docs/
|   |— architecture.md
|   |— modbus-registers.md
|   |— mqtt-topics.md
|   |— troubleshooting.md
```

Zusätzliche sinnvolle Komponenten

averaging.py

Saubere Trennung für gleitende Mittelwerte.

Beispiel:

```
from collections import deque

class MovingAverage:

    def __init__(self, size: int):
        self.values = deque(maxlen=size)

    def add(self, value: float):
        self.values.append(value)

    @property
    def average(self) -> float:
        if not self.values:
            return 0.0
        return sum(self.values) / len(self.values)
```

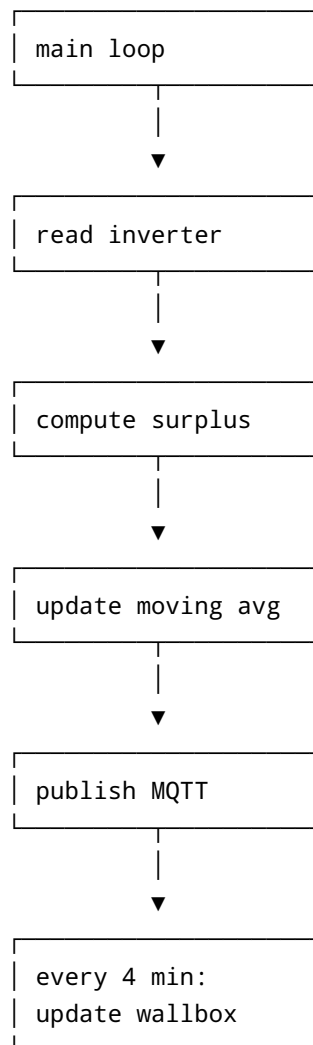
logging_setup.py

Zentrale Logging-Konfiguration.

```
import logging

def setup_logging(level="INFO"):
    logging.basicConfig(
        level=level,
        format="%(asctime)s %(levelname)s %(name)s %(message)s"
    )
```

Ablaufdiagramm



Best Guess – SolaX Modbus Register

WICHTIG:

Die Register unterscheiden sich je nach:

- X3 Generation
- Firmware
- Pocket WiFi/LAN Adapter
- Hybrid vs. Non-Hybrid

Deshalb sollten die Register konfigurierbar bleiben.

Wahrscheinlich relevante Register

Wert	Register	Typ	Einheit
PV Power Total	0x0006	uint16	W
Load Power	0x000A	int16	W
Grid Power	0x0046	int16	W
Battery Power	0x0016	int16	W
SOC	0x001C	uint16	%

Empfohlener Minimalumfang:

```
registers:  
  pv_power: 6  
  house_power: 10
```

Optional:

```
registers:  
  grid_power: 70
```

Dann kann später präziser geregelt werden.

Empfohlene Regelstrategie

Polling

Alle 20 Sekunden.

Mittelwertfenster

12 Werte.

=> 4 Minuten Mittelwert.

Wallbox-Update

Alle 240 Sekunden.

Stromberechnung

Einphasig:

$$I = P / 230V$$

Beispiele:

Leistung	Strom
1400 W	6 A
1840 W	8 A
2300 W	10 A
3680 W	16 A

MQTT Topics

```
pvcharge/pv_power  
pvcharge/house_power  
pvcharge/surplus_power  
pvcharge/surplus_average  
pvcharge/charger/enabled  
pvcharge/charger/current_limit  
pvcharge/charger/state
```

Beispiel systemd-Service

```
[Unit]  
Description=PV Charge Control  
After=network.target
```

```
[Service]
Type=simple
User=pi
WorkingDirectory=/opt/pv-charge-control
ExecStart=/usr/bin/python3 main.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

requirements.txt

```
pymodbus>=3.0
requests>=2.31
PyYAML>=6.0
paho-mqtt>=1.6
```

Download-/Dokumentations-Links

SolaX Modbus

Offizielle Modbus Dokumentation

<https://www.solaxpower.com/support/>

Suche nach:

- "SolaX Modbus Protocol"
 - "X3 Hybrid Modbus"
-

Community Modbus Register

<https://github.com/wills106/homsassistant-solax-modbus>

Sehr wertvoll für reale Registerbelegungen.

pymodbus

<https://github.com/pymodbus-dev/pymodbus>

Dokumentation:

<https://pymodbus.readthedocs.io/>

Easee

Easee API

<https://developer.easee.com/>

API Doku:

https://developer.easee.com/reference/get_api-chargers-id

Easee Python SDKs / Community

https://github.com/nordicopen/easee_hass

https://github.com/fondberg/easee_hass

Sehr hilfreich für:

- OAuth
 - Charger Control
 - State Handling
-

MQTT

Eclipse Mosquitto

<https://mosquitto.org/>

MQTT Explorer

<https://mqtt-explorer.com/>

paho-mqtt

<https://github.com/eclipse-paho/paho.mqtt.python>

Raspberry Pi / systemd

systemd Service Guide

<https://www.freedesktop.org/software/systemd/man/systemd.service.html>

Sinnvolle Erweiterungen später

Optional später ergänzbar

- Web UI
 - Home Assistant Discovery
 - Grafana Export
 - SQLite Historie
 - EV-Ladeplanung
 - Wetterprognose
 - Strompreisoptimierung
 - Batteriespeicherintegration
 - dynamische Phasenumschaltung
 - Docker Packaging
 - Prometheus Exporter
-

Architekturentscheidungen (bewusst)

Verwendet

- synchrones Polling
- single-threaded
- YAML Config
- journald Logging
- MQTT optional
- systemd

Bewusst NICHT verwendet

- asyncio
 - Flask/FastAPI
 - Kubernetes
 - Datenbank
 - Redis
 - RabbitMQ
 - komplexe Event-Systeme
 - Websocket-Architektur
-

Empfehlung zur Entwicklung

Reihenfolge

1. Modbus Read stabilisieren
 2. Moving Average testen
 3. MQTT hinzufügen
 4. Easee Cloud anbinden
 5. lokale Easee API evaluieren
 6. Regelalgorithmus tunen
 7. systemd integrieren
-

Empfehlung für lokale Easee-Integration

Die lokale Easee-Steuerung sollte experimentell bleiben.

Empfohlene Priorität:

1. Cloud API stabil implementieren
2. Interface sauber abstrahieren
3. lokale API später ergänzen

Dadurch bleibt das System robust, auch wenn Easee lokale APIs ändert.